

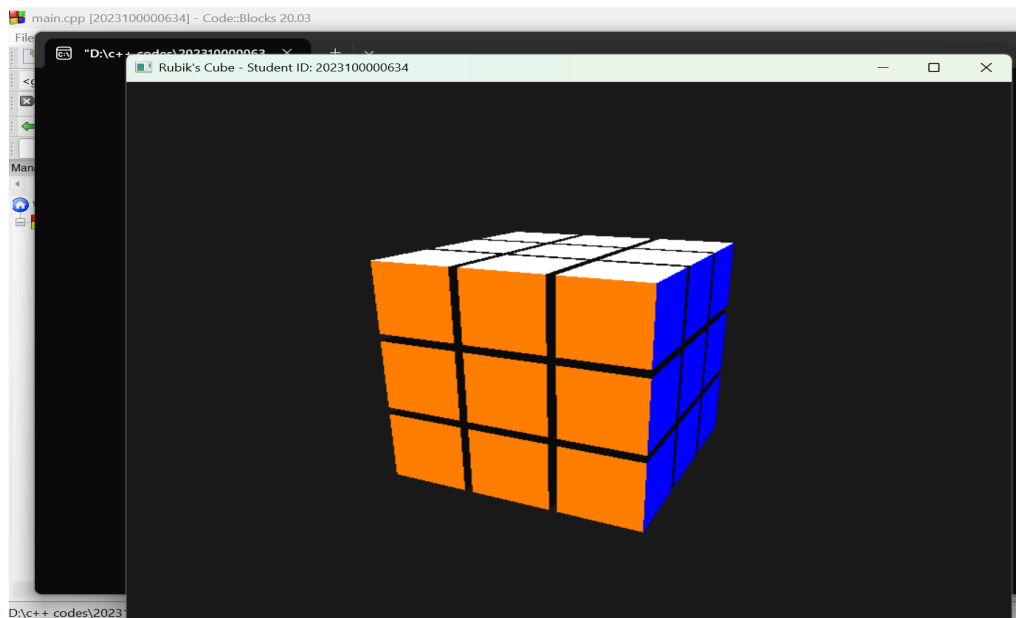
# Rubik's Cube Design Using Only Cubes

Student ID: 2023100000634

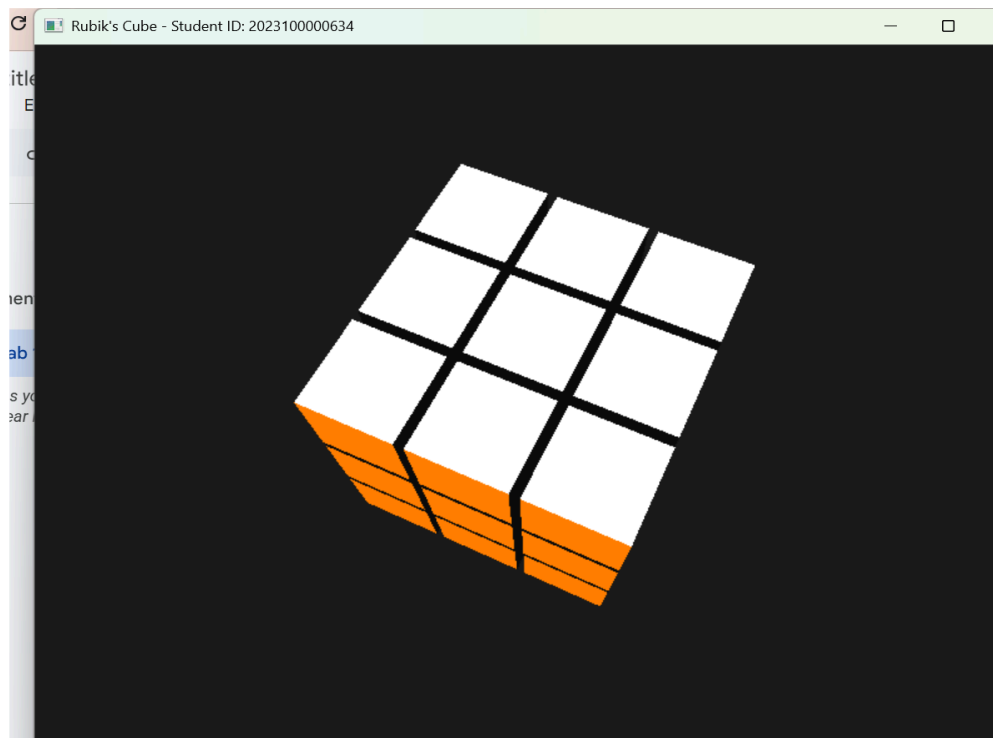
## Keyboard Controls :

|                           |               |
|---------------------------|---------------|
| Rotation along X axis     | A / B         |
| Rotation along Y axis     | C / D         |
| Rotation along Z axis     | E / F         |
| Translation X axis        | G / H         |
| Translation Y axis        | I / J         |
| Translation Z axis        | K / L         |
| Scaling / Zoom in-out     | + / -         |
| Student ID as window name | 2023100000634 |

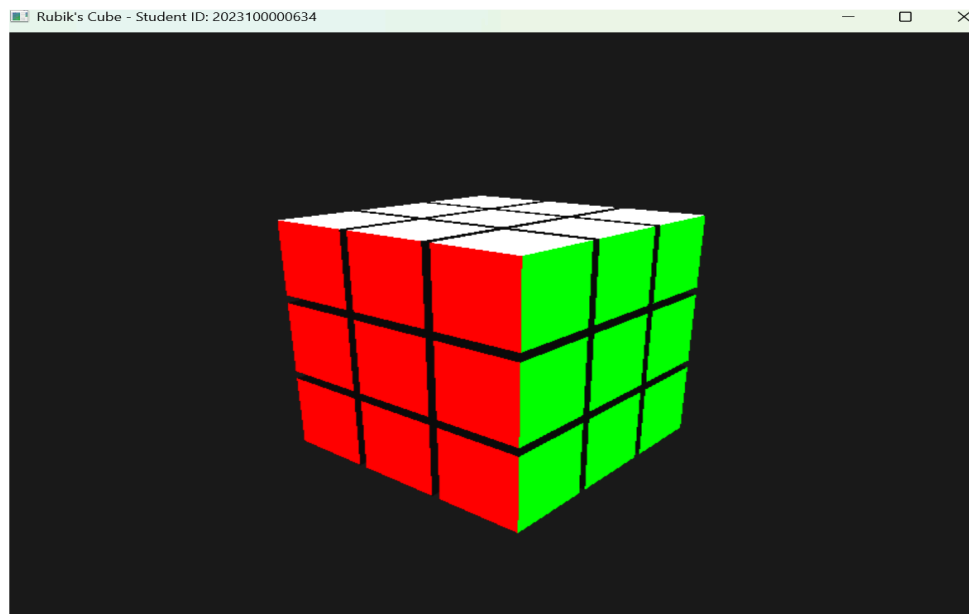
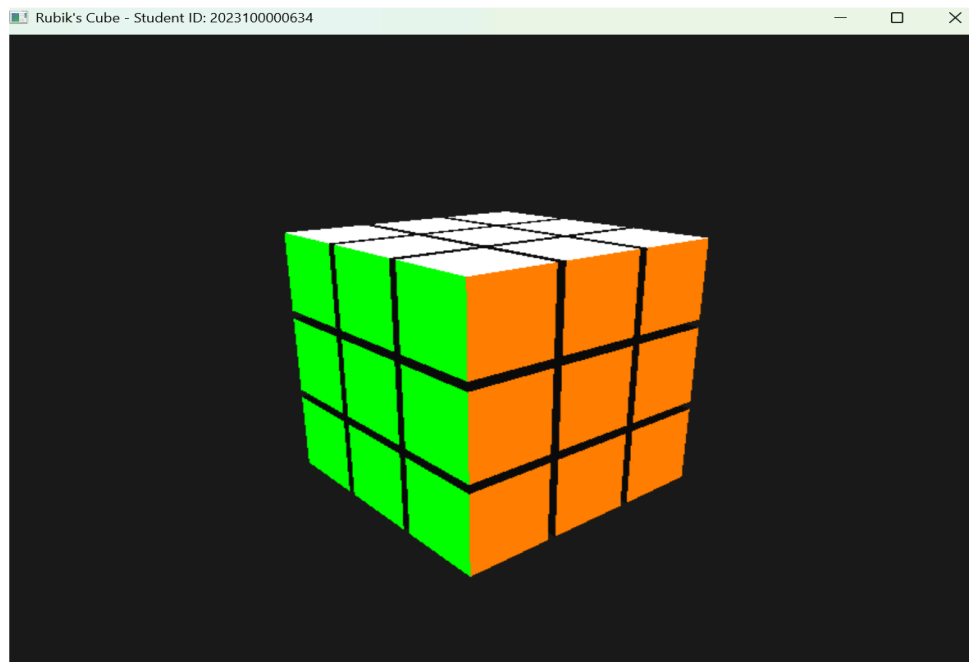
## Output Screenshot — Initial Rubik's Cube



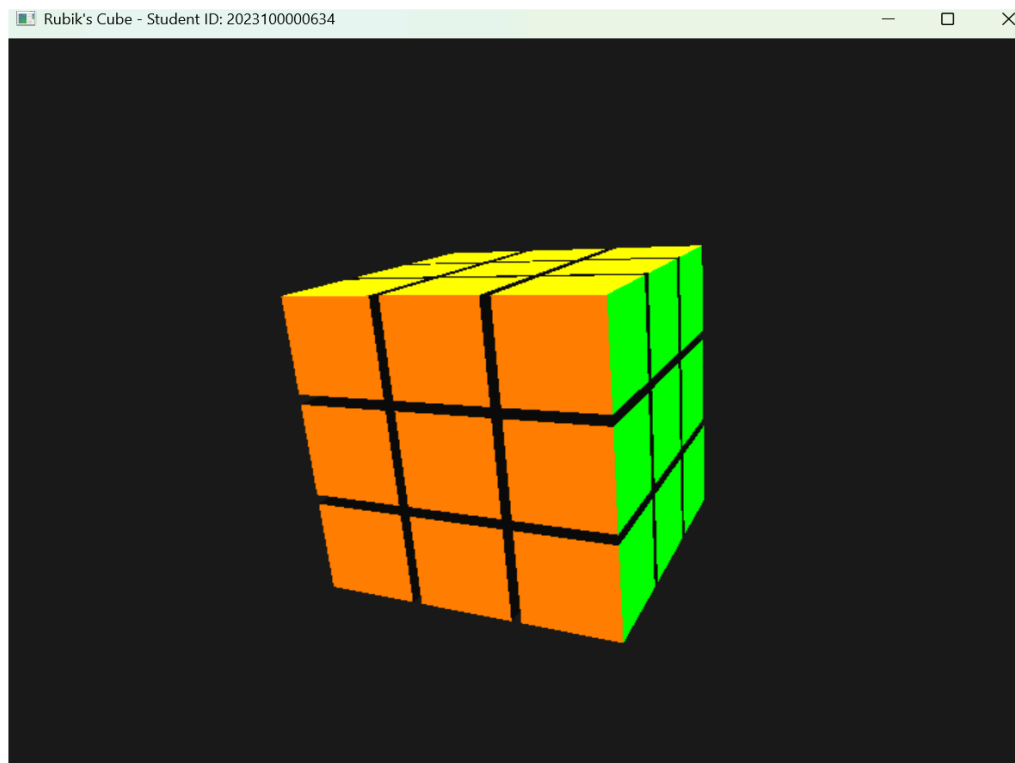
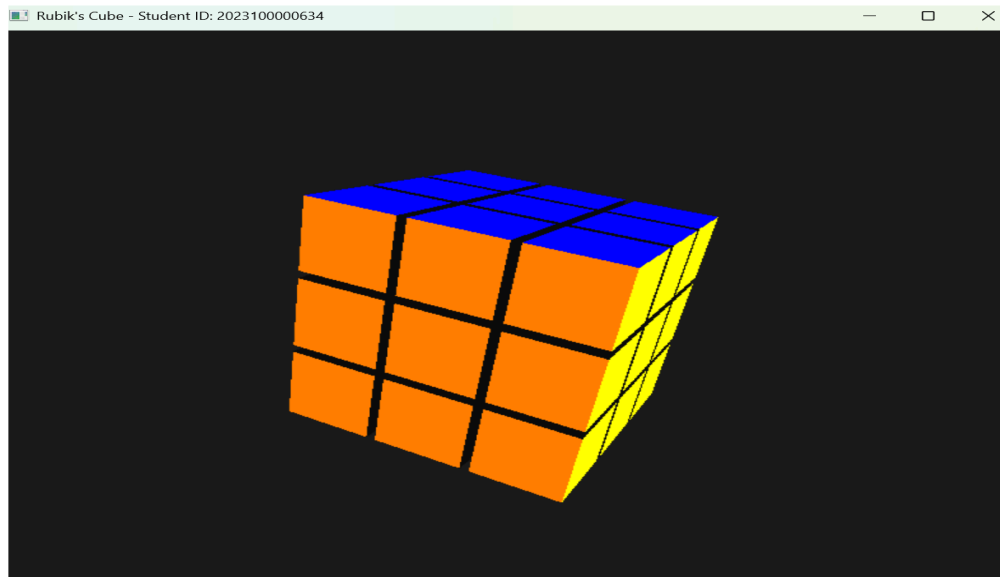
## Rotation along X axis(A / B)



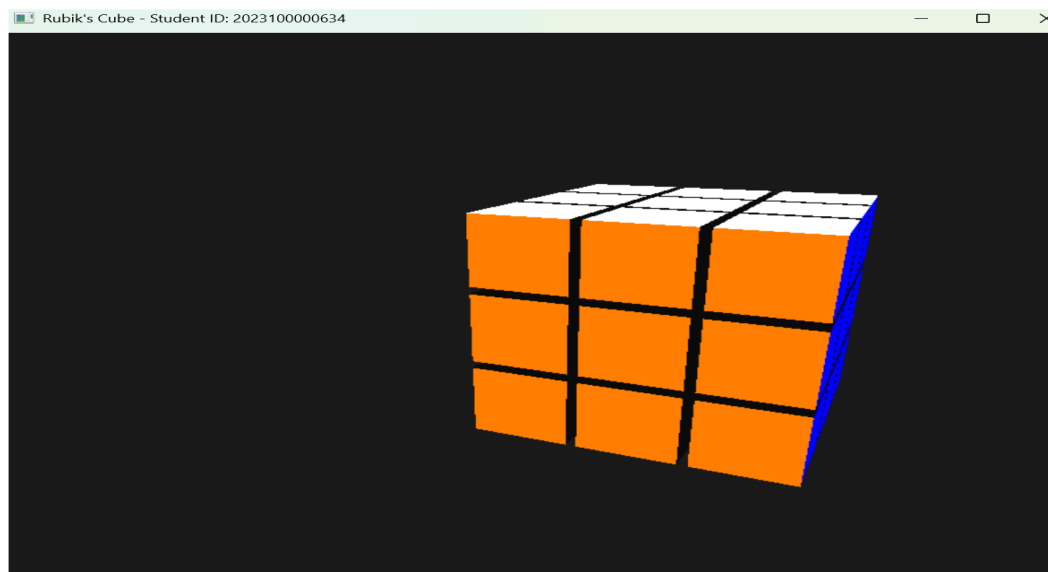
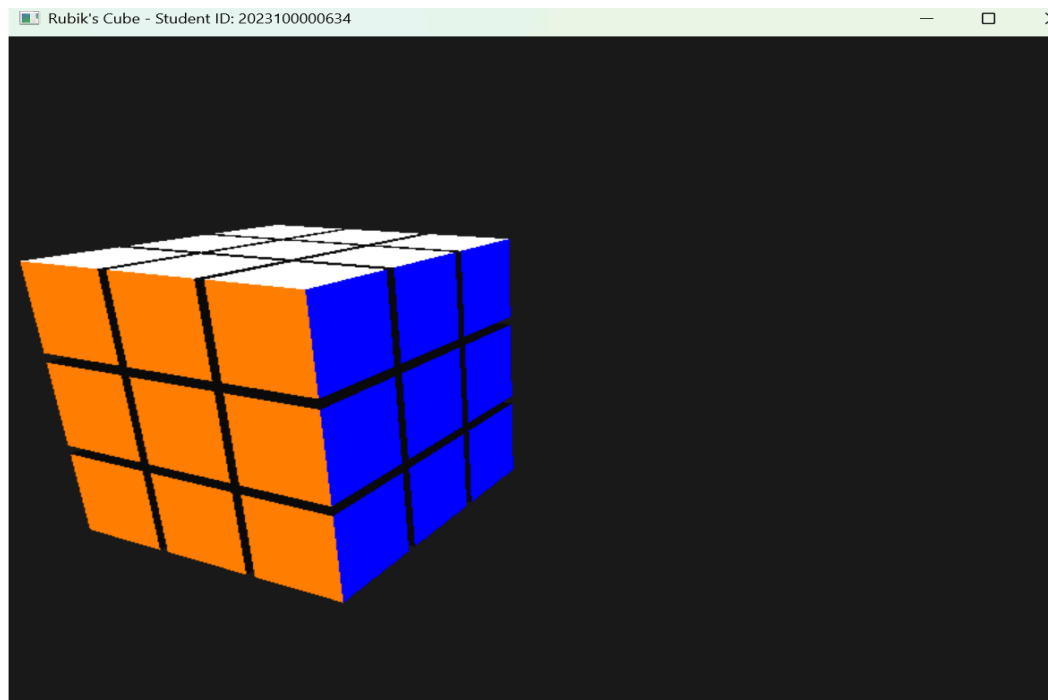
## Rotation along Y-axis(C/ D)



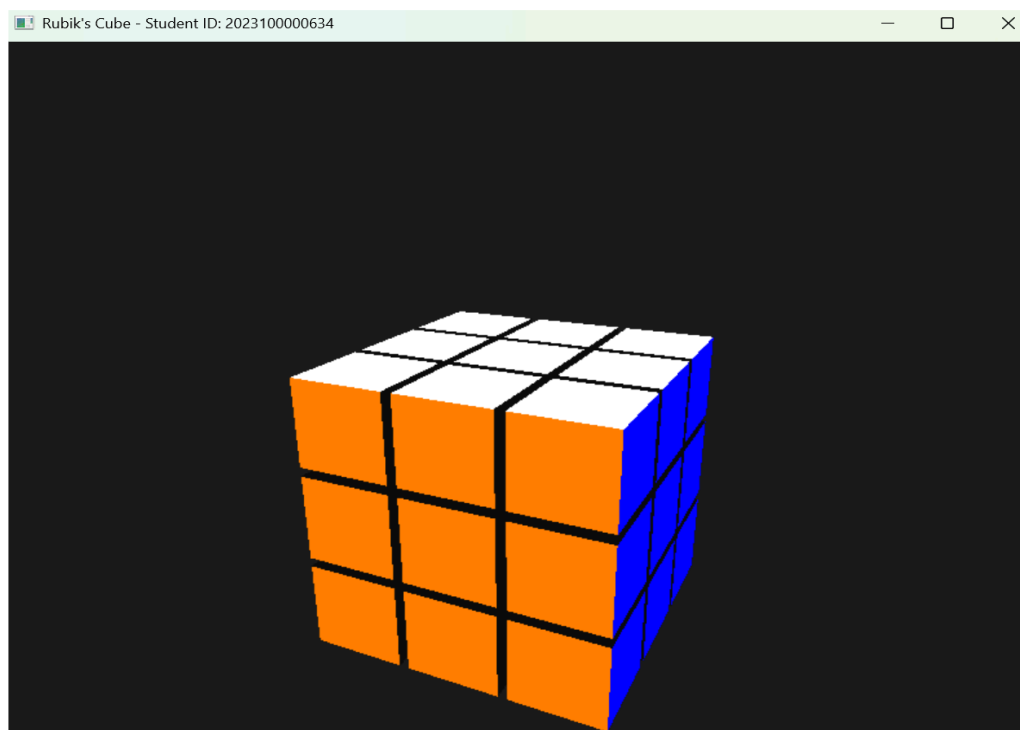
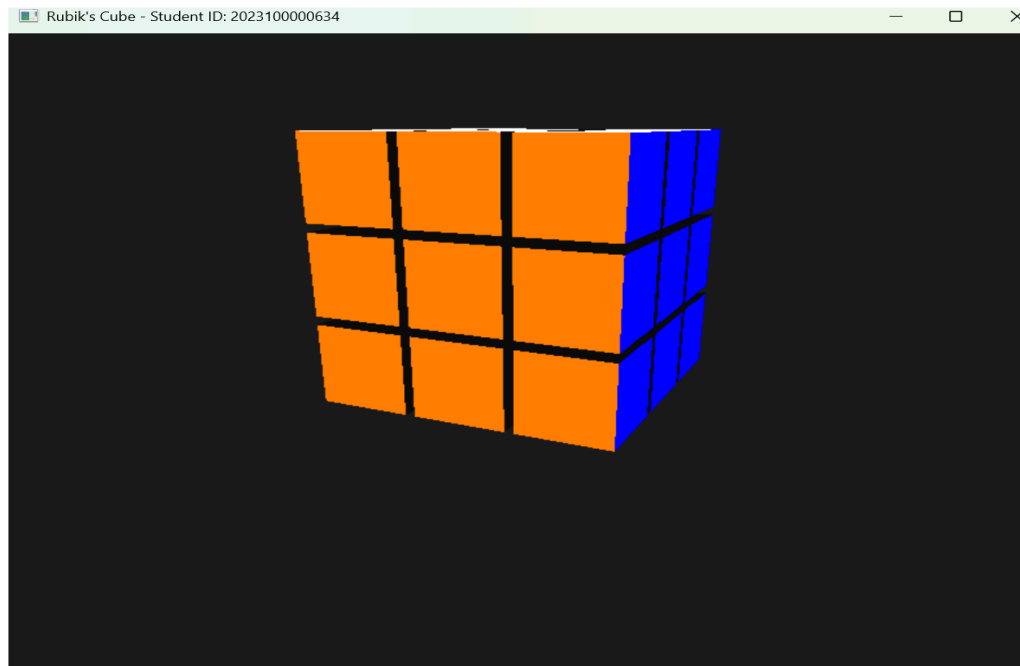
## Rotation along Z-axis(E / F)



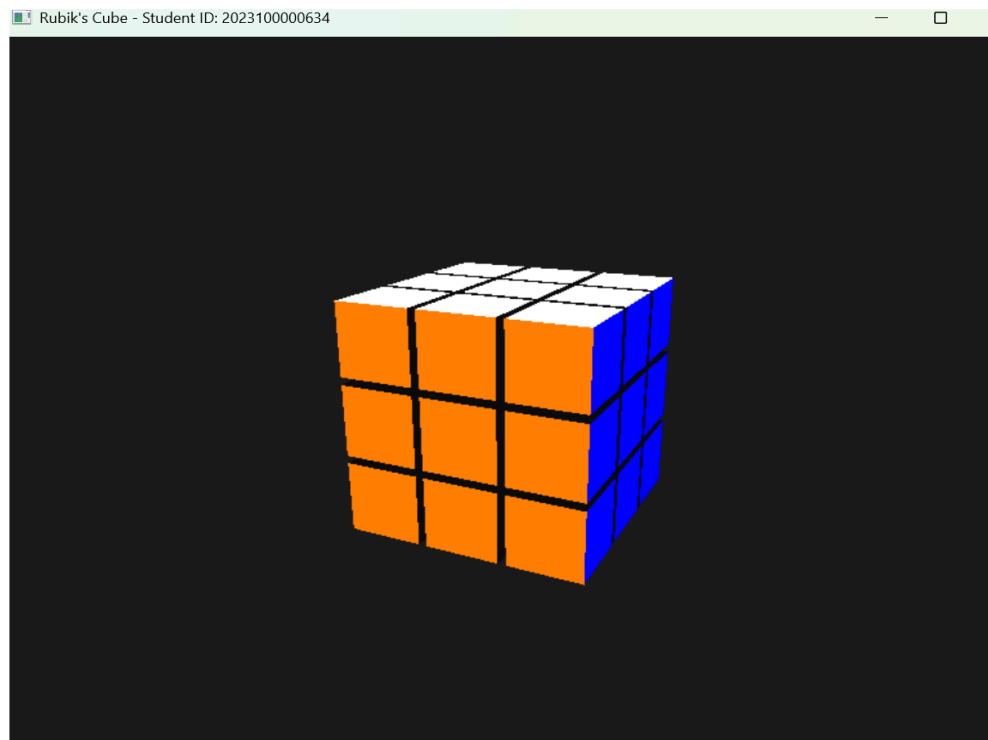
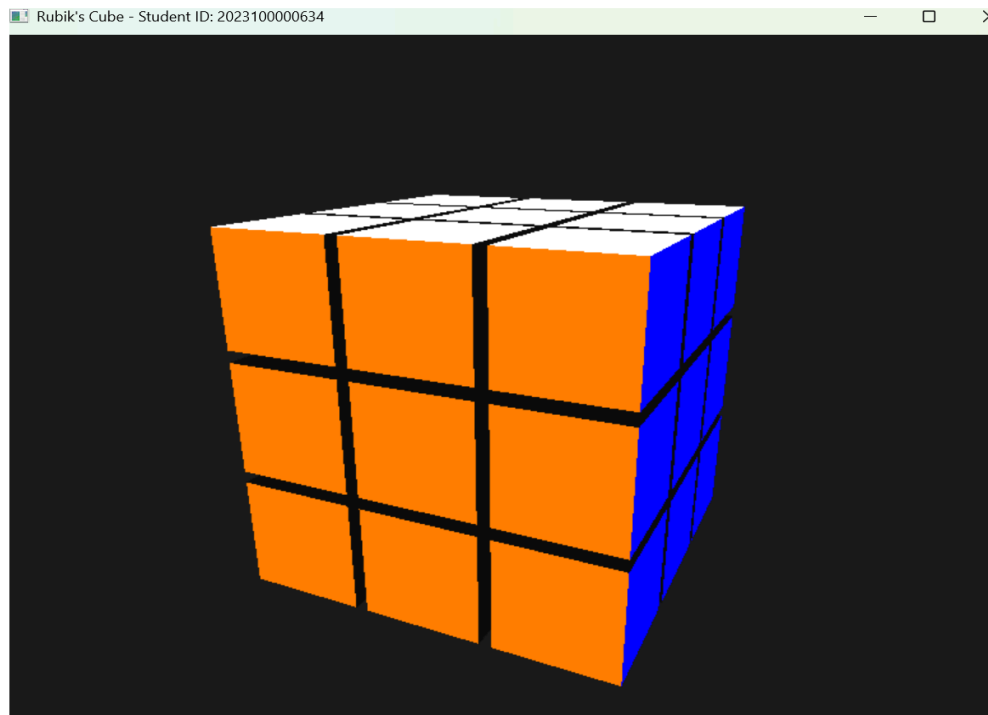
## Translation X axis(G / H)



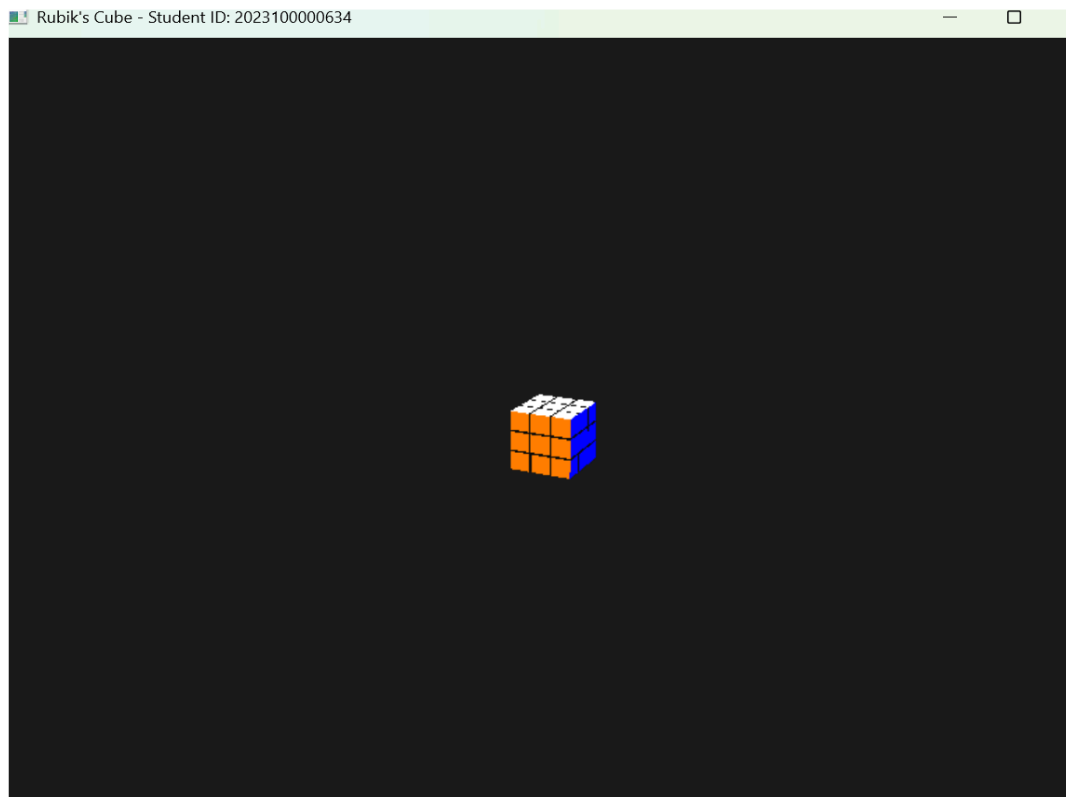
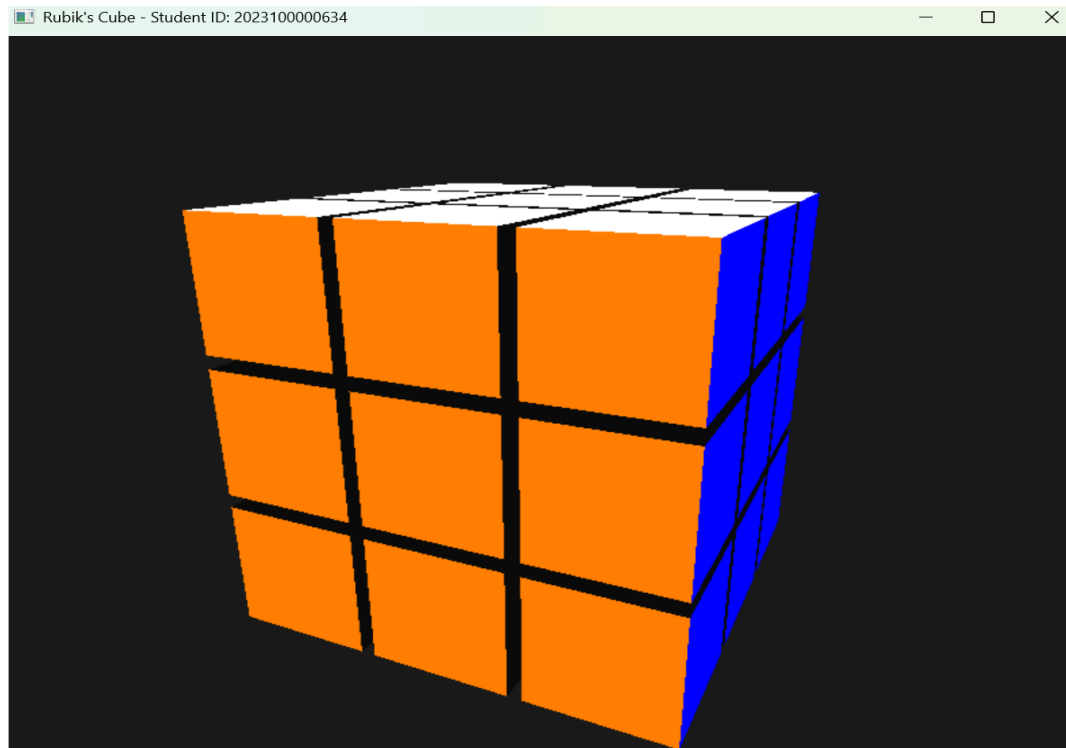
## Translation y-axis(I/ J)



## Translation Z- axis( $k / L$ )



## Scaling / Zoom in-out(+ / -)



## Source Code :

```
#include <windows.h>

#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif

#include <stdlib.h>

float degreeX = 20.0f;
float degreeY = -25.0f;
float degreeZ = 0.0f;

float moveX = 0.0f;
float moveY = 0.0f;
float moveZ = 0.0f;

float cubeScale = 1.0f;

static void resize(int width, int height)
{
    if (height == 0)
        height = 1;

    float ar = (float)width / (float)height;

    glViewport(0, 0, width, height);

    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    glFrustum(-ar, ar, -1.0, 1.0, 2.0, 100.0);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}
```

```
void cube(int x, int y, int z)
{
    glBegin(GL_QUADS);

    if (z == -1)
    {
        glColor3f(1.0f, 0.0f, 0.0f);

        glVertex3f(0.0f, 1.0f, 0.0f);
        glVertex3f(1.0f, 1.0f, 0.0f);
        glVertex3f(1.0f, 0.0f, 0.0f);
        glVertex3f(0.0f, 0.0f, 0.0f);
    }

    if (z == 1)
    {
        glColor3f(1.0f, 0.5f, 0.0f);

        glVertex3f(1.0f, 1.0f, 1.0f);
        glVertex3f(0.0f, 1.0f, 1.0f);
        glVertex3f(0.0f, 0.0f, 1.0f);
        glVertex3f(1.0f, 0.0f, 1.0f);
    }

    if (x == -1)
    {
        glColor3f(0.0f, 1.0f, 0.0f);

        glVertex3f(0.0f, 1.0f, 1.0f);
        glVertex3f(0.0f, 1.0f, 0.0f);
        glVertex3f(0.0f, 0.0f, 0.0f);
        glVertex3f(0.0f, 0.0f, 1.0f);
    }
}
```

```
if (x == 1)
{
    glColor3f(0.0f, 0.0f, 1.0f);

    glVertex3f(1.0f, 1.0f, 0.0f);
    glVertex3f(1.0f, 1.0f, 1.0f);
    glVertex3f(1.0f, 0.0f, 1.0f);
    glVertex3f(1.0f, 0.0f, 0.0f);
}
```

```
if (y == -1)
{
    glColor3f(1.0f, 1.0f, 0.0f);

    glVertex3f(1.0f, 0.0f, 1.0f);
    glVertex3f(0.0f, 0.0f, 1.0f);
    glVertex3f(0.0f, 0.0f, 0.0f);
    glVertex3f(1.0f, 0.0f, 0.0f);
}
```

```
if (y == 1)
{
    glColor3f(1.0f, 1.0f, 1.0f);

    glVertex3f(1.0f, 1.0f, 0.0f);
    glVertex3f(0.0f, 1.0f, 0.0f);
    glVertex3f(0.0f, 1.0f, 1.0f);
    glVertex3f(1.0f, 1.0f, 1.0f);
}
```

```
glEnd();
```

```
glColor3f(0.05f, 0.05f, 0.05f);
```

```
glBegin(GL_QUADS);
```

```
if (z != -1)
{
    glVertex3f(0.0f, 1.0f, 0.0f);
    glVertex3f(1.0f, 1.0f, 0.0f);
    glVertex3f(1.0f, 0.0f, 0.0f);
    glVertex3f(0.0f, 0.0f, 0.0f);
}
```

```
if (z != 1)
{
    glVertex3f(1.0f, 1.0f, 1.0f);
    glVertex3f(0.0f, 1.0f, 1.0f);
    glVertex3f(0.0f, 0.0f, 1.0f);
    glVertex3f(1.0f, 0.0f, 1.0f);
}
```

```
if (x != -1)
{
    glVertex3f(0.0f, 1.0f, 1.0f);
    glVertex3f(0.0f, 1.0f, 0.0f);
    glVertex3f(0.0f, 0.0f, 0.0f);
    glVertex3f(0.0f, 0.0f, 1.0f);
}
```

```
if (x != 1)
{
    glVertex3f(1.0f, 1.0f, 0.0f);
    glVertex3f(1.0f, 1.0f, 1.0f);
    glVertex3f(1.0f, 0.0f, 1.0f);
    glVertex3f(1.0f, 0.0f, 0.0f);
}
```

```
if (y != -1)
{
```

```
    glVertex3f(1.0f, 0.0f, 1.0f);
    glVertex3f(0.0f, 0.0f, 1.0f);
    glVertex3f(0.0f, 0.0f, 0.0f);
    glVertex3f(1.0f, 0.0f, 0.0f);
}
```

```
if (y != 1)
{
    glVertex3f(1.0f, 1.0f, 0.0f);
    glVertex3f(0.0f, 1.0f, 0.0f);
    glVertex3f(0.0f, 1.0f, 1.0f);
    glVertex3f(1.0f, 1.0f, 1.0f);
}
```

```
glEnd();
}
```

```
void rubiksCube()
{
    int x, y, z;

    for (x = -1; x <= 1; x++)
    {
        for (y = -1; y <= 1; y++)
        {
            for (z = -1; z <= 1; z++)
            {
                glPushMatrix();

                glTranslatef(
                    x * 1.05f,
                    y * 1.05f,
                    z * 1.05f
                );

                glScalef(
                    0.95f,
                    0.95f,
```

```

        0.95f
    );

    glTranslatef(
        -0.5f,
        -0.5f,
        -0.5f
    );

    cube(x, y, z);

    glPopMatrix();
}
}
}
}

static void display(void)
{
    glClear(
        GL_COLOR_BUFFER_BIT |
        GL_DEPTH_BUFFER_BIT
    );

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();

    glPushMatrix();

    glTranslatef(
        0.0f,
        0.0f,
        -8.0f
    );

    glTranslatef(
        moveX,

```

```
    moveY,  
    moveZ  
);
```

```
glScalef(  
    cubeScale,  
    cubeScale,  
    cubeScale  
);
```

```
glRotatef(  
    degreeX,  
    1.0f,  
    0.0f,  
    0.0f  
);
```

```
glRotatef(  
    degreeY,  
    0.0f,  
    1.0f,  
    0.0f  
);
```

```
glRotatef(  
    degreeZ,  
    0.0f,  
    0.0f,  
    1.0f  
);
```

```
rubiksCube();
```

```
glPopMatrix();
```

```
glutSwapBuffers();
```

```
}
```

```
static void key(unsigned char key, int x, int y)
```

```
{
```

```
    switch (key)
```

```
    {
```

```
        case 27:
```

```
        case 'q':
```

```
        case 'Q':
```

```
            exit(0);
```

```
            break;
```

```
        case 'a':
```

```
            degreeX += 5.0f;
```

```
            break;
```

```
        case 'b':
```

```
            degreeX -= 5.0f;
```

```
            break;
```

```
        case 'c':
```

```
            degreeY += 5.0f;
```

```
            break;
```

```
        case 'd':
```

```
            degreeY -= 5.0f;
```

```
            break;
```

```
        case 'e':
```

```
            degreeZ += 5.0f;
```

```
            break;
```

```
        case 'f':
```

```
            degreeZ -= 5.0f;
```

```
            break;
```

```
        case 'g':
```

```
            moveX -= 0.20f;
```

```
        break;

    case 'h':
        moveX += 0.20f;
        break;

    case 'i':
        moveY += 0.20f;
        break;

    case 'j':
        moveY -= 0.20f;
        break;

    case 'k':
        moveZ += 0.20f;
        break;

    case 'l':
        moveZ -= 0.20f;
        break;

    case '+':
    case '=':
        cubeScale += 0.10f;
        break;

    case '-':
    case '_':
        cubeScale -= 0.10f;

        if (cubeScale < 0.20f)
            cubeScale = 0.20f;

        break;

    case 'r':
    case 'R':

        degreeX = 20.0f;
```

```

        degreeY = -25.0f;
        degreeZ = 0.0f;

        moveX = 0.0f;
        moveY = 0.0f;
        moveZ = 0.0f;

        cubeScale = 1.0f;

        break;
    }

    glutPostRedisplay();
}

int main(int argc, char *argv[])
{
    glutInit(&argc, argv);

    glutInitWindowSize(
        800,
        600
    );

    glutInitWindowPosition(
        100,
        50
    );

    glutInitDisplayMode(
        GLUT_RGB |
        GLUT_DOUBLE |
        GLUT_DEPTH
    );

    glutCreateWindow(
        "Rubik's Cube - Student ID: 2023100000634"
    );

```

```
glutReshapeFunc(resize);

glutDisplayFunc(display);

glutKeyboardFunc(key);


glClearColor(
    0.10f,
    0.10f,
    0.10f,
    1.0f
);

glEnable(GL_DEPTH_TEST);


glutMainLoop();

return EXIT_SUCCESS;
}
```